



Análise e Desenvolvimento de Sistemas

**ALGORITMOS E ESTRUTURA DE
DADOS EM PYTHON**

RELATÓRIO DE AULAS PRÁTICAS

Nome: Vitória Freitas RA: 2523279

Polo de matrícula: Vila Dirce

Local da realização da Aula Prática: Alphaville - Santana de Parnaíba

Ano da postagem: 2025

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): IMPLEMENTANDO ALGORITMOS EM PYTHON

O que é lógica de programação e sua importância

É a capacidade organizar as ideias, para que possamos passar as corretas instruções e comandos, no qual farão o computador realizar uma tarefa, assim solucionando algum problema que temos que resolver. Podemos quebrar o problema em partes pequenas, resolvendo ele de pouco em pouco.

Ela é importante, pois com ela os desenvolvedores podem construir códigos mais legíveis e eficientes.

pseudocódigo e fluxograma

Pseudocódigo é uma linguagem intermediária entre a nossa linguagem natural e a de programação, usada para descrever os algoritmos. O fluxograma é um diagrama usado para descrever visualmente um algoritmo, representando uma sequência de operações.

vantagens de usar Python para aprender programação

Entre as diversas vantagens de se aprender Python, uma delas é seu fácil aprendizado, sendo ótima para iniciantes, por conta de sua sintaxe mais simples de se entender e sua extensa biblioteca, facilitando a construção de algoritmos.

Definição de algoritmo, variável, condicional, laço

Um algoritmo pode ser definido como instruções em sequência, que são bem definidas e finitas, são usados para resolver problemas ou realizar alguma tarefa.

Variável é o nome dado para os valores que vão ser manipulados e utilizados durante a operação de um programa.

Uma estrutura condicional é uma estrutura que se uma determinada condição for atendida o algoritmo vai uma decisão. Um laço, também conhecido como estrutura de repetição, é utilizado para repetir uma tarefa, até que a condição seja atendida.

Código fonte das atividades do roteiro e realizadas em sala de aula:

Exercício 1:

```
vezes = int(input("Quantas vezes você vai digitar: "))
```

```

for i in range(1, vezes + 1):
    num = int(input("Digite um numero: "))
    if(num % 2 == 0):
        print(f"{num} é par.")
    else:
        print(f"{num} é impar")

```

Exercício 2:

```

nome = input("Seu nome: ").strip()
# fazendo verificação, não se pode dividir por zero
while True:
    qtd = int(input("Quantas valores você vai digitar: "))
    if qtd != 0:
        break
    else:
        print("Insira um número maior que zero")

soma = 0.0

for i in range(qtd):
    valor = float(input("Valor: "))
    soma = soma + valor

media = soma / qtd

if media >= 70:
    status = "Excelente"
elif 50 <= media <= 69:
    status = "Bom"
else:
    status = "Precisa melhorar"

print("\n ---- resultado ---- ")
print(f"Aluno(a): {nome}")
print(f"Media: {media:.2f}")
print(f"Classificação: {status}")

```

Referências

<https://www.locaweb.com.br/blog/temas/codigo-aberto/logica-de-programacao-o-que-e/>

<https://www.facom.ufu.br/~backes/gci007/Aula02-AlgoritmosFluxogramas.pdf>

<https://king.host/blog/tecnologia/python/>

<https://blog.geekhunter.com.br/o-que-e-algoritmo/>

<https://gaea.com.br/variaveis-programacao/>

<https://www.treinaweb.com.br/blog/estruturas-condicionais-e-de-repeticao>

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ESTRUTURAS DE DADOS LINEARES EM PYTHON: LISTAS, PILHAS, FILAS E EFICIÊNCIA. NOTAÇÃO BIG-O

Definição de listas, pilhas, filas e introdução à notação Big-O

Listas na programação é uma estrutura de dados que são usadas para armazenar um conjunto de dados em uma única variável.

Pilhas também são uma estrutura de dados, elas guardam o valor de maneira sequencial, como se estivesse empilhando um acima do outro, onde o último a entrar é primeiro a sair (LIFO, Last In First Out).

Filas, por outro lado, são seguem o padrão FIFO (First In First Out), no qual o primeiro elemento a ser inserido é primeiro a sair.

A notação Big-O descreve a complexidade de um algoritmo em relação ao tamanho da entrada, utilizando termos algébricos. Por exemplo, $O(n^2)$, com n sendo o tamanho da entrada.

Código fonte das atividades do roteiro e realizadas em sala de aula:

```
from collections import deque

fila = deque()

fila.append("Cliente A")
fila.append("Cliente B")
fila.append("Cliente C")
# utilizando o popleft, pois é mais eficiente que o pop(0)
atendido = fila.popleft()

print("Atendido: ", atendido)
print("fila agora: ", list(fila))

fila.append("Cliente D")
print("Proximo a ser atendido", fila[0])
```

```
# atidade realizada para ver a diferença no tempo de execução do pop(0) e
o popleft()
from collections import deque
import time
```

```
n = 20000

lst = list(range(n))
t0 = time.time()

while lst:
    lst.pop(0)

t1 = time.time()
tempo_list = t1 - t0

dq = deque(range(n))
t0 = time.time()

while dq:
    dq.popleft()

t1 = time.time()
tempo_deque = t1 - t0

print(f"Tempo list.pop(0): {tempo_list:.3f} s")
print(f"Tempo deque.popleft: {tempo_deque:.3f} s")
```

```
from collections import deque

pilha = []
pilha.append("prato 1")
pilha.append("prato 2")
print(pilha.pop())

fila = deque()
fila.append("cliente 1")
fila.append("cliente 2")
print(fila.popleft())
```

```
import time
```

```

fila = deque()
pilha = []
t0 = time.time()
for i in range(3):
    nome = input("Nome do cliente: ")
    fila.append(nome)

print("\n Iniciando atendimentos.")

while fila:
    cliente = fila.popleft()
    print(f"Atendendo {cliente}")

for i in range(3):
    tarefa = input(f"Tarefa {i+1}: ")
    pilha.append(tarefa)

print(f"\nExecutando tarefa {i+1}")

while pilha:
    print(f"Executando: {pilha.pop()}")
t1 = time.time()

tempo_de_execucao = t1 - t0
print("Tempo de execução: ", tempo_de_execucao)

```

Referências

<https://cursos.alura.com.br/forum/topico-explicacao-o-que-sao-listas-317465>
<https://www.treinaweb.com.br/blog/o-que-e-e-como-funciona-a-estrutura-de-dados-pilha>
<https://www.treinaweb.com.br/blog/o-que-e-e-como-funciona-a-estrutura-de-dados-fila>
<https://www.freecodecamp.org/portuguese/news/o-que-e-a-notacao-big-o-complexidade-de-tempo-e-de-espaco/>

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ESTRUTURAS DE DADOS NÃO LINEARES - ÁRVORES E GRAFOS EM PYTHON

ORIENTAÇÕES:

Cada aluno deve produzir um relatório curto (2 a 3 páginas), contendo:

Resumo Teórico:

- Definição de árvore e grafo, vantagens de cada estrutura.
- Comentário sobre a escolha de Python para ilustrar algoritmos de percursos;

Códigos Desenvolvidos:

- Inserir o *código-fonte completo* da atividade proposta.
- Comentar as principais linhas, ressaltando criação de vértices, filas, laços e condições de parada.

Conclusão:

- Descrever se a aula ajudou no entendimento de árvores e grafos.

REFERÊNCIAS: O aluno deverá colocar o nome dos livros e *sites* utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resumo Teórico	2,0	Clareza e correlação correta entre conceitos.
Organização e Comentários do Código	3,0	O código deve estar indentado corretamente, usar nomes de variáveis adequados e conter comentários informativos (quando necessários).
Funcionalidade e do Código	2,0	Execução sem erros, entrada e saída corretas.
Criatividade e Aprimoramentos	3,0	Inclusão de pesos, tratamento de exceções ou visualização gráfica simples.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ALGORITMOS DE ORDENAÇÃO EM PYTHON (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort)

ORIENTAÇÕES:

Cada aluno deve produzir um relatório sintético (2 a 3 páginas) contendo:

Resumo Teórico:

- Definir ordenação e justificar sua relevância em ciência da computação.
- Explicar diferenças conceituais entre algoritmos quadráticos e log-lineares.
- Comentar vantagens e limitações de cada método;

Códigos Desenvolvidos:

- Inserir as implementações completas, destacando linhas decisivas (trocas, partições, fusões).
- Incluir tabela dos tempos obtidos nas medições.

REFERÊNCIAS: O aluno deverá colocar o nome dos livros e *sites* utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resum o Teórico	2,0	Clareza e correlação correta entre conceitos.
Organização e Comentários do Código	3,0	O código deve estar indentado corretamente, usar nomes de variáveis adequados e conter comentários informativos (quando necessários).
Funcionalidade do Código	3,0	Execução correta e apresentação dos resultados de tempo.
Criatividade e Aprimoramentos	2,0	Introdução de visualização gráfica simples, pivô aleatório ou otimizações.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ALGORITMOS DE PESQUISA - BUSCA LINEAR E BUSCA BINÁRIA EM PYTHON

ORIENTAÇÕES:

Cada aluno deve produzir um relatório (2 a 3 páginas) contendo:

Resumo Teórico:

- Explicar a diferença entre pesquisa exaustiva e pesquisa por divisão.
- Comparar custos de busca linear e binária em termos de complexidade e de requisitos de ordenação;

Códigos Desenvolvidos:

- Inserir implementações completas de ambos os métodos.
- Apresentar tabela com tempos coletados para três tamanhos distintos de listas.

REFERÊNCIAS: O aluno deverá colocar o nome dos livros e *sites* utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resumo Teórico	2,0	Clareza conceitual e uso correto de terminologia.
Organização e Comentários do Código	3,0	O código deve estar indentado corretamente, usar nomes de variáveis adequados e conter comentários informativos (quando necessários).
Funcionalidade do Código	3,0	Execução correta e apresentação dos resultados de tempo.
Criatividade e Aprimoramentos	2,0	Execução correta e apresentação dos resultados de tempo.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): TABELAS DE DISPERSÃO (HASH TABLES) E OS HEAPS EM PYTHON

ORIENTAÇÕES:

Cada aluno deve produzir um relatório (2 a 3 páginas) contendo:

Resumo Teórico:

- Definir tabelas de dispersão, explicar colisões e tratamentos.
- Descrever heaps binários e justificar eficiência em filas de prioridade;

Códigos Desenvolvidos:

- Inserir implementações das medições solicitadas, com observações sobre linhas-chave (cálculo de hash, heappush, heappop).
- Apresentar tabela dos tempos obtidos em cada experimento.

REFERÊNCIAS: O aluno deverá colocar o nome dos livros e *sites* utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resumo Teórico	2,0	Precisão conceitual e clareza de exposição.
Organização e Comentários do Código	3,0	O código deve estar indentado corretamente, usar nomes de variáveis adequados e conter comentários informativos (quando necessários).
Funcionalidade do Código	3,0	Execução sem erros, coleta e exibição confiável dos tempos.
Criatividade e Aprimoramentos	2,0	Implementação de heap de máx-prioridade, visualizações simples ou análise de fator de carga.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): ALGORITMOS DE GRAFOS - DIJKSTRA, BELLMAN-FORD, KRUSKAL E PRIM EM PYTHON

ORIENTAÇÕES:

Cada aluno deve produzir um relatório (2 a 3 páginas) contendo:

Resumo Teórico:

- Explicar diferenças entre caminhos mínimos de fonte única e árvores geradoras mínimas.
- Apontar condições de aplicabilidade (pesos negativos, denso × esperso);

Códigos Desenvolvidos:

- Incluir implementações completas, indicando linhas de relaxamento e união-busca.
- Apresentar tabela de tempos e pesos totais das árvores.

REFERÊNCIAS: O aluno deverá colocar o nome dos livros e *sites* utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resum o Teórico	2,0	Precisão conceitual e clareza de exposição.
Organização e Comentários do Código	3,0	O código deve estar indentado corretamente, usar nomes de variáveis adequados e conter comentários informativos (quando necessários).
Funcionalidade do Código	3,0	Execução sem erros, resultados coerentes.
Criatividade e Aprimoramentos	2,0	Uso de visualizações, análise de ciclos negativos ou comparação com bibliotecas externas.

TÍTULO DA ATIVIDADE (ROTEIRO OU AULA): TÉCNICAS DE DIVISÃO E CONQUISTA E DE PROGRAMAÇÃO DINÂMICA EM PYTHON

ORIENTAÇÕES:

Cada aluno deve produzir um relatório (2 a 3 páginas) contendo:

Resumo Teórico:

- A explanação das diferenças estruturais entre divisão e conquista e programação dinâmica.
- A justificativa dos ganhos obtidos com memoização ou tabulação nos problemas escolhidos;

Descrição das Classes Criadas:

- A inclusão das três versões do algoritmo escolhido (recursiva simples, memoizada, bottom-up).
- A apresentação dos tempos medidos em tabela.

REFERÊNCIAS: O aluno deverá colocar o nome dos livros e *sites* utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resumo Teórico	2,0	A inclusão das três versões do algoritmo escolhido (recursiva simples, memoizada, bottom-up).
Organização e Comentários do Código	3,0	A apresentação dos tempos medidos em tabela.
Funcionalidade do Código	3,0	A inclusão das três versões do algoritmo escolhido (recursiva simples, memoizada, bottom-up).
Criatividade e Aprimoramentos	2,0	Análise gráfica, discussão sobre consumo de memória ou casos extremos..

